Universidad de La Sabana · Observabilidad en Ambientes Productivos

LAB integrador: sistema de observabilidad end-to-end con AIOps y resiliencia en GCP y AWS

Reporte ejecutivo final

Beycy Yuliet Rojas Acero · Manuel Santofimio

Repositorio: github.com/yuliet719-beep/lab-observabilidad-e2eN (tag v1.0)

Octubre de 2026

1. Resumen ejecutivo

El presente documento reporta la integración de los dominios de observabilidad (tres pilares, OpenTelemetry, AIOps, observabilidad de red y seguridad, DataOps y SRE) en un sistema de tres microservicios desplegado en una **producción simulada multi-nube**. Al no contar el equipo con cuentas de GCP ni de AWS, la solución se ejecutó en un sandbox reproducible que emula la topología de ambas nubes (VPC, mesh, bases de datos gestionadas y flujos de red), y la infraestructura de nube real quedó definida como código Terraform validado (*terraform validate*) pero no aplicado. Esta decisión cumple, además, la recomendación de ejecutar los experimentos de caos únicamente en sandbox.

- **Detección (MTTD):** E2 (10 % de errores en data-service) se detectó en 76 s. E1 (latencia de 200 ms en service-b) notificó a los 16 s, pero la regla ya estaba en pending por un pico transitorio previo a la inyección; la estimación conservadora (re-disparo atribuible solo a E1) es de 106 s. Ambos cumplen el objetivo de menos de 2 minutos.
- **Alertas accionables:** cada página llegó con el trace_id de un request afectado y una causa raíz candidata (service-b en E1 y data-service en E2). En ambos casos la causa coincidió con la falla inyectada.
- **Reducción de ruido:** en 12 minutos sin fallas reales, los umbrales estáticos generaron 11 notificaciones y el pipeline AIOps 0 (−100,0 %).
- **SLO y error budget:** E1 llevó la proporción de requests bajo 300 ms a 90,8 % (objetivo 99 %; burn rate 9,2×). E2 bajó la disponibilidad a 81,11 % (objetivo 99,5 %; burn rate 37,8×).
- **Seguridad:** el escenario de ataque activó 2 tipos de alerta (fuerza bruta, escaneo de puertos y flujo Este-Oeste no permitido hacia la base de datos), con su hallazgo normalizado tipo SCC/Security Hub. Además se reportaron 0 CVE críticos y 141 altos activos en las imágenes.

2. Arquitectura observable completa (Módulo A)

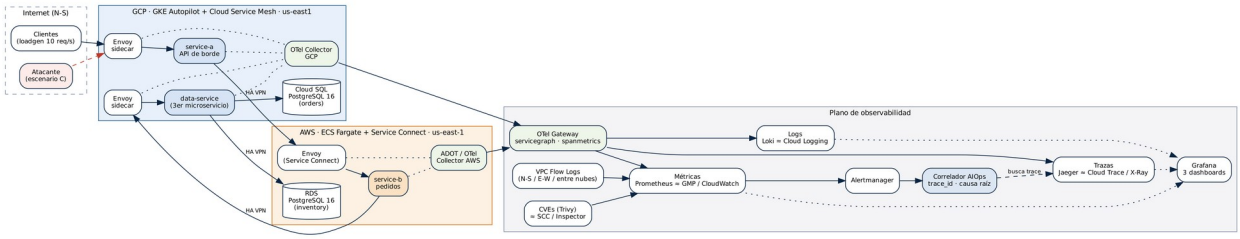


Figura 1. Arquitectura desplegada. Las líneas punteadas son exportación OTLP; «≈» indica el servicio gestionado equivalente en la nube.

La arquitectura extiende el laboratorio 2.2 con un tercer microservicio, **data-service**, que consulta dos motores gestionados: Cloud SQL for PostgreSQL (pedidos, GCP) y Amazon RDS for PostgreSQL (inventario, AWS). El flujo Norte-Sur entra por service-a (GKE), atraviesa la interconexión hacia service-b (ECS Fargate) y regresa a data-service. Los tres servicios usan el SDK de OpenTelemetry para los tres pilares: trazas OTLP, métricas (histograma *http.server.request.duration* con buckets alineados al SLO) y logs con trace_id/span_id. Los spans de base de datos siguen las **OTel Database Semantic Conventions** estables (*db.system.name*, *db.namespace*, *db.query.text*, *db.operation.name*, *error.type*) y propagan el contexto a Cloud SQL Query Insights mediante sqlcommenter.

El **service mesh L7** se implementó con sidecars Envoy 1.33, que emiten métricas por clase de respuesta, spans propios y access logs vía OTLP, y que admiten inyección de fallas en caliente. En la nube, el equivalente es Cloud Service Mesh gestionado en GCP y **ECS Service Connect** en AWS: AWS App Mesh llegó a fin de soporte el 30 de septiembre de 2026 (Amazon Web Services [AWS], 2024). Un collector por nube agrega atributos *cloud.provider/platform/region* y reenvía a un gateway central que deriva el grafo de servicios (servicegraph) y las métricas RED de las trazas. Así, una misma traza cruza ambas nubes y conserva la correlación completa.

Tabla 1. Equivalencia entre el sandbox ejecutado y el IaC de nube (validado)

Capacidad	Sandbox (ejecutado)	GCP (infra/gcp)	AWS (infra/aws)
Cómputo y mesh	Contenedores + Envoy sidecar	GKE Autopilot + Cloud Service Mesh	ECS Fargate + Service Connect

Capacidad	Sandbox (ejecutado)	GCP (infra/gcp)	AWS (infra/aws)
Datos	PostgreSQL 16 ×2	Cloud SQL PG16 + Query Insights	RDS PG16 + Performance Insights
Trazas / métricas / logs	Jaeger / Prometheus / Loki	Cloud Trace / GMP / Cloud Logging	X-Ray / CloudWatch / CloudWatch Logs
AIOps	Reglas ±2σ + correlador	Alertas PromQL + forecast	ANOMALY_DETECTION_BAND + alarma compuesta
Red y seguridad	Flow logs (conntrack) + Trivy	VPC Flow Logs, firewall logs, SCC	VPC Flow Logs v5, Security Hub, GuardDuty, Inspector
Caos	Envoy runtime + /admin/chaos	VirtualService fault	AWS FIS con stop condition

3. AIOps: detección automática de anomalías y correlación (Módulo B)

La detección se basa en líneas base dinámicas: para cada servicio se graban la media y la desviación estándar del error ratio y del p99 en una ventana móvil (30 min con un desfase de 5 min, para que la propia anomalía no contamine la base). La **regla de correlación** exigida dispara cuando **error_rate > baseline + 2σ Y latency_p99 > SLO_threshold (300 ms)** se sostiene durante 30 s. Como la tasa de error de base es casi nula, σ también lo es; por eso la banda tiene un piso de 0,5 %, igual al presupuesto del SLO de disponibilidad (99,5 %). Una segunda regla cubre las degradaciones sin errores (p99 fuera de banda, sobre el SLO y con menos del 99 % de requests bajo 300 ms). La banda ±2σ sobre data-service replica la detección de anomalías nativa (CloudWatch *ANOMALY_DETECTION_BAND* y el pronóstico de Cloud Monitoring). DevOps Guru no se usó porque deja de admitir clientes nuevos el 29 de octubre de 2026 (AWS, 2026).

Alertmanager entrega cada página al **correlador**. Este busca en Jaeger el trace más reciente fallido (o lento) del servicio, identifica la causa raíz candidata (el span con error más profundo o el de mayor tiempo propio), la contrasta con las aristas del servicegraph, adjunta los cambios recientes (despliegues y experimentos) y agrupa las alertas en incidentes. El resultado es una **alerta enriquecida con el trace_id del request fallido**.

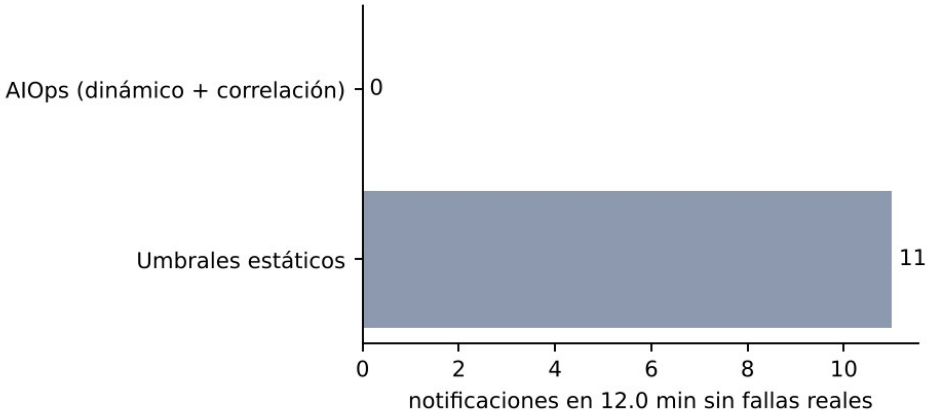


Figura 2. Notificaciones durante 12 min de operación con variabilidad natural (micro-cortes y job batch) y sin fallas inyectadas.

Tabla 2. Ruido: umbrales estáticos frente a AIOps (misma ventana y mismo group_by de Alertmanager)

Métrica	Umbrales estáticos	AIOps (dinámico + correlación)
Notificaciones	11	0
Instancias de alerta en firing	19	0
Proyección por día	1.320	0
Detalle	StaticHighErrorRate: 4 StaticHighLatencyP99: 5 StaticMeshUpstream5xx: 2	—

4. Observabilidad de red y seguridad (Módulo C)

Se habilitaron flow logs en ambas VPC. En la nube se usan VPC Flow Logs de GCP (intervalo de 5 s, muestreo del 50 %, metadatos completos) y de AWS (formato v5 con *flow-direction*, *traffic-path*, *ecs-service-name*). En el sandbox, un emulador lee la tabla conntrack del kernel y produce registros en ambos formatos. Cada flujo se clasifica en **Norte-Sur, Este-Oeste o entre nubes** y se contrasta con una **matriz de tráfico permitido**. Las alertas cubren flujos Este-Oeste no permitidos, picos de flujos REJECT (escaneo de puertos), tráfico entre nubes que triplica su línea base y fuerza bruta de autenticación (fallas > línea base + 3σ). Los hallazgos se normalizan en formato OCSF (Security Hub) con su categoría equivalente en Security Command Center.

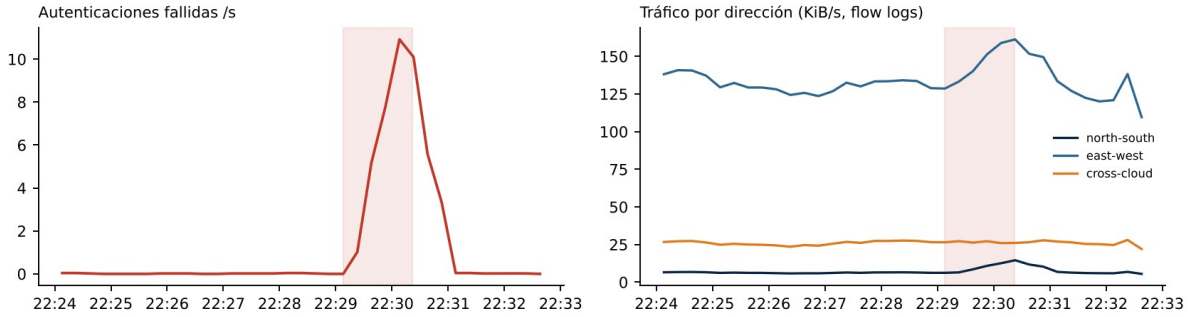


Figura 3. Golden Signals de Seguridad durante el escenario de ataque (sombreado): autenticaciones fallidas y tráfico por dirección.

Tabla 3. Detección del escenario de ataque (fuerza bruta → escaneo E-W → movimiento lateral a Cloud SQL)

Alerta	Origen	t desde el inicio	Resumen
SecUnexpectedEastWestFlow	unknown	52 s	Flujo E-W no permitido unknown -> mesh-service-a:10000
SecAuthBruteForce	—	52 s	Intentos de autenticación fallidos anómalos (7.345/s)
SecUnexpectedEastWestFlow	unknown	112 s	Flujo E-W no permitido unknown -> mesh-service-a:10000
SecUnexpectedEastWestFlow	unknown	112 s	Flujo E-W no permitido unknown -> cloudsql-orders:5432
SecUnexpectedEastWestFlow	unknown	112 s	Flujo E-W no permitido unknown -> data-service:8000

Golden Signals de Seguridad (dashboard Grafana y su equivalente en Cloud Monitoring y CloudWatch): (1) intentos de autenticación fallidos, con un pico de 10,9/s frente a ~0,02/s en la base; (2) tráfico N-S/E-W/entre nubes, con flujos no permitidos hacia Cloud SQL:5432 y hacia data-service:8000 (bypass del mesh) detectados a los 52–112 s. El escaneo de puertos no produjo registros REJECT (0): los intentos contra puertos cerrados no persistieron en conntrack entre lecturas de 2 s, brecha que se corrige en el roadmap con eBPF; (3) CVEs activos (Trivy sobre las tres imágenes): 0 críticos, 141 altos y 198 medios, la mayoría en paquetes del sistema base de Debian.

5. Chaos engineering controlado y validación del MTTD (Módulo D)

Se ejecutaron dos experimentos con hipótesis, alcance acotado al sandbox y un guardrail automático (se aborta si la disponibilidad de service-a cae por debajo del 80 % durante 60 s). **E1:** latencia de 200 ms en el 100 % de los requests a service-b, inyectada en su sidecar Envoy (equivalente a AWS FIS *aws:ecs:task-network-latency*). **E2:** error rate del 10 % en data-service con fallas reales de base de datos (consultas que exceden *statement_timeout*). La variante con abort del mesh queda en el IaC.

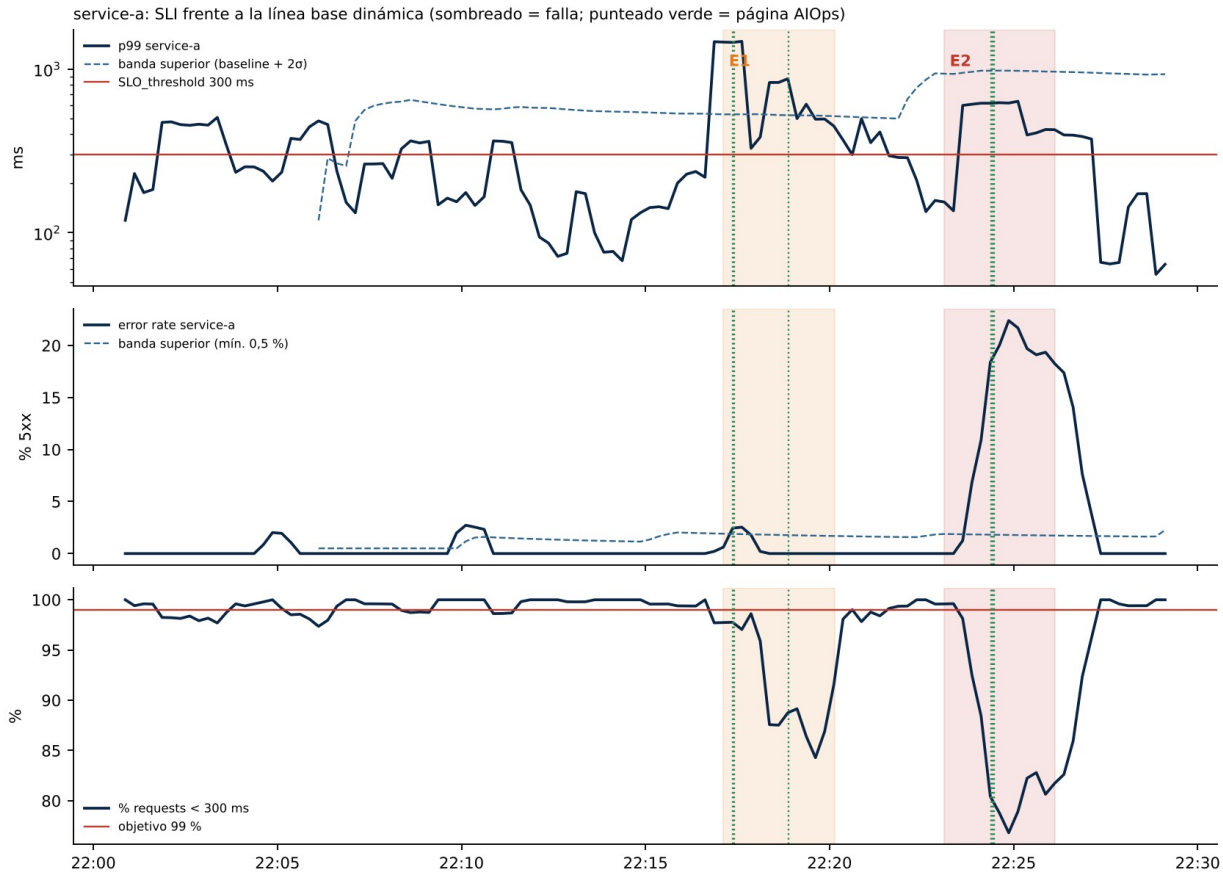


Figura 4. SLI de service-a frente a la línea base dinámica. Sombreado: falla activa; línea punteada verde: página AIOps recibida.

Tabla 4. Resultados por experimento (SLO 28 días: disponibilidad 99,5 %; latencia 99 % < 300 ms)

Pregunta / métrica	E1 · +200 ms en service-b	E2 · 10 % errores en data-service
MTTD (inyección → notificación)	16 s nominal; 106 s conservador (pending previo por pico transitorio)	76 s (firing: 70 s)
¿MTTD < 2 min?	Sí	Sí
Primera alerta	AIOpsLatencyAnomalySLO	AIOpsCorrelatedErrorLatency
¿Se degradó el SLO?	Sí (latencia): 90,8 % < 300 ms; p99 pico 1.474 ms	Sí (disponibilidad): 81,11 %; error pico 22,4 %
Burn rate durante la falla	9,2× (latencia)	37,8× (disponibilidad)
Error budget 28 d consumido	0,07 % del de latencia	0,28 % del de disponibilidad
¿Alerta accionable?	Sí: trace d7f7a9eb8551…, causa service-b (ingress)	Sí: trace d5ed37fd74e1…, causa data-service (SELECT)
Ruido estático en paralelo	9 notificaciones	9 notificaciones

Análisis. El error budget mensual absorbió ambos experimentos (0,07 % y 0,28 %) porque la falla duró 3 minutos. Sin embargo, la velocidad de consumo es la señal relevante: con el burn rate observado, el presupuesto de latencia se agotaría en ~73,0 h y el de disponibilidad en ~17,8 h. Esto justifica paginar (y no abrir un ticket) en ambos casos. Las alertas fueron accionables: el operador recibe el trace del request afectado y el salto responsable (ingress en service-b; SELECT en data-service). Esto corrige la falla de atribución observada en el Game Day anterior, donde el correlador culpó al servicio que llamaba en lugar del dependiente.

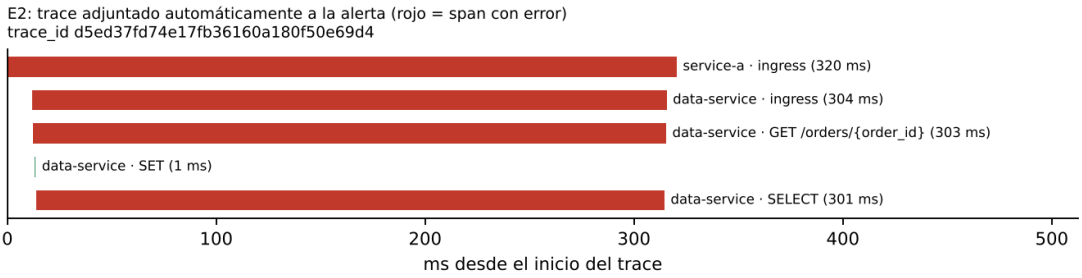


Figura 5. Trace adjuntado a la alerta de E2: el span de BD en data-service (rojo) propaga el error hasta service-a.

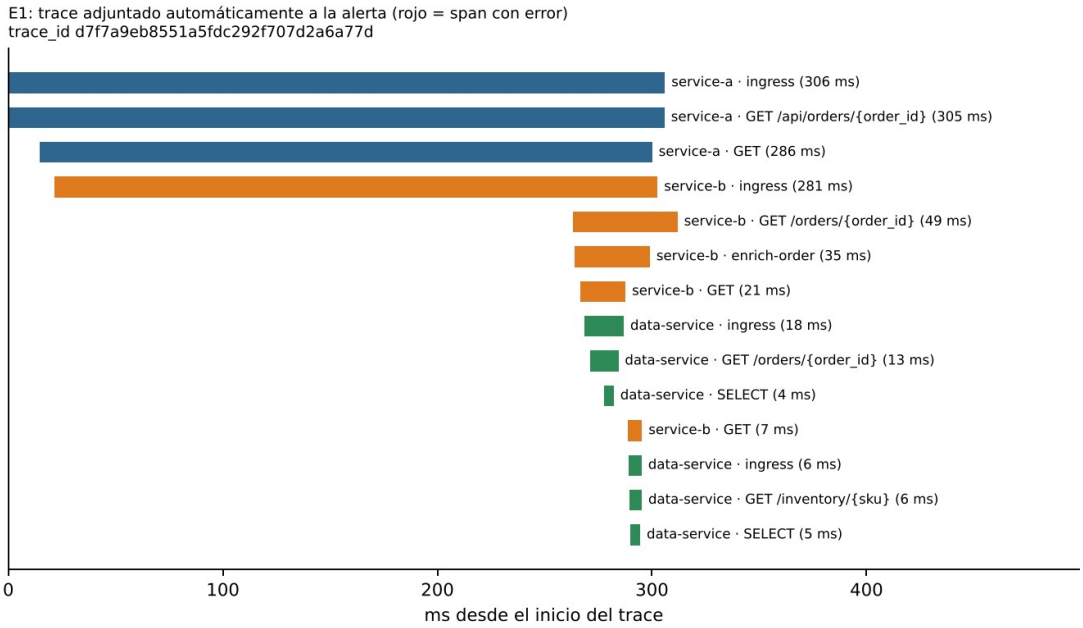


Figura 6. Trace adjuntado a la alerta de E1: el span ingress del sidecar de service-b concentra ~200 ms de tiempo propio (falla inyectada en el mesh).

Evidencia: alerta enriquecida entregada al operador (E2)

Tabla 7. Campos de la notificación generada por el correlador (extracto de docs/evidencias)

Campo	Valor
alertname / servicio	AIOpsCorrelatedErrorLatency / data-service
incident_id	INC-1792380265
trace_id	130fa61c6803acbee2a0e4bda5bd141f
causa raíz candidata	{"service":"data-service","span":"SELECT","error.type":null,"db.system.name":"postgresql","db.query.text":"SELECT pg_sleep(1)"}
aristas servicegraph con fallas	user->service-a: 11.6%; service-a->service-b: 14.3%; service-b->data-service: 8.1%; data-service->postgresql: 5.4%
aristas lentas	user->service-a: 616ms; service-a->service-b: 499ms; service-b->data-service: 479ms
cambios recientes	E1 start: latencia +200 ms en service-b (Envoy fault injection, 100 %); E1 end; E2 start: error rate 10 % en data-service (statement_timeout BD)
SLO al alertar	error ratio 12,1 % (banda 0,9 %), p99 396 ms
tiempo de enriquecimiento	1187 ms

La alerta incluye el contexto de cambio (el experimento en curso), lo que en producción equivale a correlacionar con el último despliegue. Este es el principal acelerador del MTTR: el operador no necesita buscar el trace ni adivinar el servicio responsable.

6. Reporte de madurez de observabilidad (Módulo E)

La solución se autoevaluó contra los ocho dominios del Observability Foundation Blueprint (DevOps Institute, s. f.) en una escala de 1 (inicial) a 5 (optimizada). La calificación se apoya en la evidencia del repositorio y no en capacidades declaradas.

Tabla 5. Autoevaluación de madurez (promedio 3,1 / 5)

Dominio	Nivel	Evidencia	Brecha principal
1. Fundamentos y cultura de observabilidad	3	SLOs explícitos, runbook y decisiones documentadas (ADR)	Sin rituales de equipo (revisión de SLO, postmortems periódicos)
2. Pilares (logs, métricas, trazas)	4	3 pilares en 3 servicios + mesh + BD, correlacionados por trace_id	Sin profiling continuo ni RUM
3. Ecosistema open source / OpenTelemetry	4	OTel SDK, collectors por nube + gateway, semconv HTTP/DB estables	Sin OpAMP ni operador de autoinstrumentación
4. Mapas de servicio y topología	3	Servicegraph con nodos virtuales de BD y aristas entre nubes	No integra topología de red/infra ni dependencias externas
5. DataOps de telemetría	2	Retención acotada y límites de CPU para el plano de observabilidad	Muestreo al 100 %, sin control de cardinalidad ni de costos
6. AIOps	3	Baseline ±2σ, correlación multiseñal, enriquecimiento e incidentes	Sin estacionalidad, sin ML, sin auto-remediación
7. Seguridad y redes	3	Flow logs N-S/E-W, golden signals de seguridad, CVEs, hallazgos OCSF	Sin SIEM/SOAR ni eBPF; escaneo de puertos no detectado (REJECT)
8. Prácticas DevOps y SRE	3	Error budget, caos con guardrails, CI de validación	Sin política automática de error budget ni burn rate multi-ventana

Roadmap a 3 meses para alcanzar el siguiente nivel

Tabla 6. Roadmap accionable (cada hito con su métrica de salida)

Mes	Acciones	Criterio de salida
Mes 1	Aplicar el IaC en proyectos sandbox de GCP y AWS con HA VPN; muestreo tail-based en el gateway (100 % de errores y lentos, 10 % del resto); presupuesto de cardinalidad.	Trazas entre nubes en Cloud Trace/X-Ray; −50 % de volumen de spans sin perder trazas con error
Mes 2	Alertas SLO multi-ventana (1 h/5 min, 6 h/30 min) y política de error budget (congelar despliegues al superar el 50 %); línea base estacional (hora del día) y servicegraph en el correlador; eBPF (Cilium/Hubble o GKE Dataplane V2) para red L3–L7.	MTTD < 60 s; ≥ 90 % de páginas con causa raíz correcta; 0 páginas sin acción en 2 semanas
Mes 3	SCC/Security Hub y GuardDuty hacia un SIEM con playbooks SOAR; Game Days mensuales con AWS FIS y fault injection del mesh; circuit breaker y load shedding en service-b; dashboards de costos de telemetría.	MTTR < 15 min en el Game Day; CVEs críticos con fix < 7 días; madurez promedio ≥ 3,8

7. Limitaciones y conclusiones

Limitaciones. (1) El despliegue en GCP y AWS no se aplicó por falta de cuentas; el IaC está validado sintácticamente y contra los esquemas de los proveedores, pero no se probó en ejecución. (2) El sandbox corre en 2 vCPU, lo que eleva la variabilidad del p99 y obligó a fijar el SLO en 300 ms. (3) El ruido de base es sintético (perfil parametrizado de micro-cortes y job batch), por lo que la reducción de alertas demuestra el mecanismo pero no predice la reducción en un sistema real. (4) Por el plazo de entrega, la ventana de comparación de ruido (12 min) y las fallas (3 min) fueron cortas; la reducción del 100 % de notificaciones en esa ventana no debe extrapolarse. (5) Un pico de latencia transitorio, ajeno al experimento, coincidió con el inicio de E1.

Conclusión. La correlación de señales (errores + latencia + trazas) sobre líneas base dinámicas permitió detectar ambas degradaciones dentro del objetivo de MTTD, con alertas que indican dónde actuar, mientras los umbrales estáticos produjeron ruido sin contexto. Las decisiones de diseño (Service Connect por el fin de App Mesh, CloudWatch Anomaly Detection por el retiro de DevOps Guru, flow logs sin impacto en el datapath) se documentan como ADR en el repositorio.

Anexo A. Inventario del repositorio (v1.0)

Ruta	Contenido
services/	service-a, service-b, data-service (FastAPI) y telemetry.py común: OTel SDK, 3 pilares, convenciones HTTP/DB estables
mesh/	Sidecars Envoy (métricas L7, trazas, access logs OTLP, fault injection en runtime)
otel/	Collectors GCP y AWS + gateway (servicegraph, spanmetrics, exportación a Jaeger/Prometheus/Loki)
observability/	Reglas SLI y baseline ±2σ, alertas estáticas/AIOps/seguridad, Alertmanager, Loki, Jaeger, 2 dashboards Grafana
aiops/correlator/	Enriquecimiento con trace_id, causa raíz, servicegraph, incidentes y hallazgos OCSF
network/flowlogs/	Emulador de VPC Flow Logs (AWS v5 y GCP vpc_flows), matriz de tráfico permitido
infra/gcp/	VPC + Flow Logs + firewall logging, GKE Autopilot, Cloud Service Mesh, Cloud SQL, alertas PromQL, métricas de logs, dashboard de seguridad, SCC → Pub/Sub
infra/aws/	VPC + Flow Logs v5, ECS Fargate + Service Connect, RDS, anomaly detection + alarma compuesta, Security Hub, GuardDuty, Inspector, FIS
infra/k8s/	Despliegues GKE, collector con exportadores de Google Cloud, reglas GMP, VirtualService de caos
chaos/, scripts/	Orquestador E1/E2 con guardrails, escenario de ataque, run_lab.sh, analyze.py, escaneo Trivy
docs/	Runbook, decisiones (ADR), madurez, guion del video y evidencias (resultados.json, figuras, flow logs, alertas)
.github/workflows/ci.yml	Validación de reglas, Alertmanager, Envoy, collectors, Terraform (fmt/validate), build y Trivy

Referencias

Amazon Web Services. (s. f.). AWS observability best practices. https://aws-observability.github.io/observability-best-practices/

Amazon Web Services. (2024). AWS App Mesh end of support. https://docs.aws.amazon.com/app-mesh/latest/userguide/

Amazon Web Services. (2026). Amazon DevOps Guru end of support. https://docs.aws.amazon.com/devops-guru/latest/userguide/devops-guru-end-of-support.html

Amazon Web Services. (s. f.). Use the AWS FIS aws:ecs:task actions. https://docs.aws.amazon.com/fis/latest/userguide/ecs-task-actions.html

Basiri, A., Behnam, N., de Rooij, R., Hochstein, L., Kosewski, L., Reynolds, J., & Rosenthal, C. (2016). Chaos engineering. IEEE Software, 33(3), 35–41. https://doi.org/10.1109/MS.2016.60

Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (2016). Site reliability engineering: How Google runs production systems. O'Reilly Media.

DevOps Institute. (s. f.). Observability Foundation. https://www.devopsinstitute.com/

Google Cloud. (s. f.). Cloud Observability documentation. https://cloud.google.com/stackdriver/docs

Google Cloud. (s. f.). Security Command Center overview. https://cloud.google.com/security-command-center/docs/concepts-security-command-center-overview

OpenTelemetry Authors. (s. f.). Semantic conventions for database calls and systems. https://opentelemetry.io/docs/specs/semconv/db/

Sigelman, B., et al. (2021). Towards observability data management at scale. ACM SIGOPS.

Tigera. (s. f.). eBPF and network observability. https://www.tigera.io/learn/guides/ebpf/